

```
#!/usr/bin/env python3
"""
Protótipo v2: Núcleo + Periféricos, com
"modo cerimônia" (REBUS) e
período refratário dirigido por HRV (dado
sintético-placeholder).

IMPORTANTE: a série de HRV usada aqui é
SINTÉTICA – um placeholder gerado
para ter a forma estatística plausível de
uma série real de RMSSD (ruído
suave, faixa 20-80ms). Assim que você tiver
dados reais (export do
Oura/Whoop/Apple Health em CSV, ou via
conector), troque a função
`load_hrv_series()` para ler o arquivo
real. O resto do código não muda.
"""

import numpy as np
import matplotlib.pyplot as plt

#
=====
=====
# PARÂMETROS FIXOS
#
=====
=====
K = 4
labels = ['Visão', 'Audição',
```

```

'Interocepção', 'Linguagem interna']
T = 400
kappa = 0.25
precision_ema = 0.05
ignition_threshold = 2.5
broadcast_gain_normal = 0.5
obs_noise = 0.3
refractory_period_default = 15
precision_ceiling_normal = 5.0

def true_signal(k, t):
    base_freqs = [0.02, 0.035, 0.015, 0.05]
    event_times = [100, 180, 260, 320]
    val = np.sin(2 * np.pi * base_freqs[k]
* t)
    if t > event_times[k]:
        val += 1.5
    return val

#
=====
=====
# HRV SINTÉTICO (PLACEHOLDER – trocar por
dado real)
#
=====
=====
def load_hrv_series(T, seed=99):
    """
    PLACEHOLDER. Gera uma série de RMSSD
sintética (20-80 ms) via passeio
aleatório suavizado, só para ter a
forma estatística de HRV real.
    SUBSTITUA esta função por leitura de

```

```

CSV real quando disponível, ex:
    df = pd.read_csv('hrv_export.csv')
    return df['rmssd'].values
"""
    rng = np.random.RandomState(seed)
    raw = rng.randn(T).cumsum()
    raw = (raw - raw.min()) / (raw.max() -
raw.min()) # normaliza [0,1]
    # suaviza (média móvel) para parecer
    HRV real, não ruído branco
    kernel = np.ones(20) / 20
    smooth = np.convolve(raw, kernel,
mode='same')
    smooth = (smooth - smooth.min()) /
(smooth.max() - smooth.min())
    return 20 + 60 * smooth # escala para
faixa realista de RMSSD (20-80ms)

def hrv_to_refractory(hrv_series,
min_ref=8, max_ref=30):
    """
    Mapeamento HRV -> período refratário
    (HIPÓTESE DE MODELO, não validada):
    HRV alto (bom tônus vagal) ->
    refratário curto (sistema recupera rápido,
    tolera reativação mais frequente
    sem cascata)
    HRV baixo -> refratário longo (sistema
    precisa de mais 'descanso' forçado
    para não entrar em ruminação)
    """
    hrv_norm = (hrv_series -
hrv_series.min()) / (hrv_series.max() -
hrv_series.min() + 1e-9)

```

```

        return np.round(max_ref - (max_ref -
min_ref) * hrv_norm).astype(int)

#
=====
=====
# SIMULAÇÃO (função reutilizável)
#
=====
=====
def run_simulation(seed=7,
ceremony_window=None, ceremony_gain=0.05,

ceremony_precision_ceiling=1.5,
                        refractory_series=None,
refractory_fixed=refractory_period_default)
:
    """
        ceremony_window: (t_start, t_end) ou
None. Durante a janela, o
        broadcast_gain e o teto de precisão
caem (REBUS: relaxamento de
        priors/precisão top-down).
        refractory_series: array de tamanho T
com período refratário por
        passo (ex: derivado de HRV), ou
None para usar refractory_fixed.
    """
    np.random.seed(seed)
    mu = np.zeros(K)
    precision = np.ones(K) * 1.0
    err_ema_sq = np.ones(K) * 1.0
    refractory_until = np.zeros(K,
dtype=int)

```

```

hist = {
    'mu': np.zeros((T, K)), 'true':
np.zeros((T, K)),
    'saliency': np.zeros((T, K)),
'ignition': np.zeros(T, dtype=bool),
    'winner': np.full(T, -1),
'abs_err': np.zeros((T, K)),
}

for t in range(T):
    in_ceremony = ceremony_window is
not None and ceremony_window[0] <= t <=
ceremony_window[1]
    bgain = ceremony_gain if
in_ceremony else broadcast_gain_normal
    p_ceiling =
ceremony_precision_ceiling if in_ceremony
else precision_ceiling_normal
    ref_period = refractory_series[t]
if refractory_series is not None else
refractory_fixed

    true_vals =
np.array([true_signal(k, t) for k in
range(K)])
    obs = true_vals +
np.random.randn(K) * obs_noise

    err = obs - mu
    saliency = precision * err**2
    mu = mu + kappa * precision * err
    err_ema_sq = (1 - precision_ema) *
err_ema_sq + precision_ema * err**2

```

```

        precision = np.clip(1.0 / (0.1 +
err_ema_sq), 0.1, p_ceiling)

        eligible = np.array([salience[k] if
t >= refractory_until[k] else -1.0 for k in
range(K)])
        winner = int(np.argmax(eligible))
        ignite = eligible[winner] >
ignition_threshold
        if ignite:
            refractory_until[winner] = t +
ref_period
            for j in range(K):
                if j != winner:
                    mu[j] = mu[j] + bgain *
(mu[winner] - mu[j])

            hist['mu'][t] = mu
            hist['true'][t] = true_vals
            hist['salience'][t] = salience
            hist['winner'][t] = winner if
ignite else -1
            hist['ignition'][t] = ignite
            hist['abs_err'][t] = np.abs(mu -
true_vals)

        return hist

#
=====
=====
# EXPERIMENTO 1: modo cerimônia (REBUS) vs.
baseline
#

```

```

=====
=====
ceremony_window = (255, 285) # sobrepõe o
evento de Interocepção em t=260

hist_baseline = run_simulation(seed=7,
ceremony_window=None)
hist_ceremony = run_simulation(seed=7,
ceremony_window=ceremony_window)

def window_stats(hist, window, label):
    lo, hi = window
    err_window = hist['abs_err']
    [lo:hi+1].mean(axis=0)
    err_after = hist['abs_err']
    [hi+1:hi+41].mean(axis=0) # 40 passos
    depois
    n_ign = hist['ignition']
    [lo:hi+40].sum()
    print(f"\n[{label}] Erro médio |mu-
true| DURANTE janela ({lo}-{hi}):")
    for k in range(K):
        print(f"    {labels[k]:20s}:
{err_window[k]:.3f}")
    print(f"[{label}] Erro médio |mu-true|
DEPOIS da janela (40 passos):")
    for k in range(K):
        print(f"    {labels[k]:20s}:
{err_after[k]:.3f}")
    print(f"[{label}] Ignições na janela +
40 passos seguintes: {n_ign}")
    return err_window, err_after, n_ign

print("=" * 70)

```

```

print("EXPERIMENTO 1: MODO CERIMÔNIA
(REBUS) vs. BASELINE")
print("=" * 70)
ew_base, ea_base, n_base =
window_stats(hist_baseline,
ceremony_window, "BASELINE (sem
cerimônia)")
ew_cer, ea_cer, n_cer =
window_stats(hist_ceremony,
ceremony_window, "COM CERIMÔNIA")

improvement = (ea_base - ea_cer) / (ea_base
+ 1e-9) * 100
print(f"\nMelhora percentual no erro pós-
janela (cerimônia vs baseline), por
módulo:")
for k in range(K):
    print(f"  {labels[k]:20s}:
{improvement[k]:+.1f}%")

#
=====
=====
# EXPERIMENTO 2: refratário fixo vs.
refratário dirigido por HRV
#
=====
=====
hrv_series = load_hrv_series(T)
refractory_from_hrv =
hrv_to_refractory(hrv_series)

hist_fixed_ref = run_simulation(seed=7,
refractory_series=None) # usa

```



```

refractory_fixed=15
hist_hrv_ref = run_simulation(seed=7,
refractory_series=refractory_from_hrv)

print("\n" + "=" * 70)
print("EXPERIMENTO 2: REFRATÁRIO FIXO vs.
REFRATÁRIO DIRIGIDO POR HRV (sintético)")
print("=" * 70)
print(f"Refratário fixo:
{refractory_period_default} passos
(constante)")
print(f"Refratário via HRV: varia entre
{refractory_from_hrv.min()} e
{refractory_from_hrv.max()} passos")
print(f"\nTotal de ignições (refratário
fixo): {hist_fixed_ref['ignition'].sum()}")
print(f"Total de ignições (refratário via
HRV): {hist_hrv_ref['ignition'].sum()}")
err_fixed_total =
hist_fixed_ref['abs_err'].mean()
err_hrv_total =
hist_hrv_ref['abs_err'].mean()
print(f"\nErro médio geral |mu-true|
(refratário fixo): {err_fixed_total:.4f}")
print(f"Erro médio geral |mu-true|
(refratário via HRV): {err_hrv_total:.4f}")

#
=====
=====

# PLOTS
#
=====
=====

```

```

fig, axes = plt.subplots(4, 1, figsize=(13,
14), sharex=False)
colors = ['tab:blue', 'tab:orange',
'tab:green', 'tab:red']

# Paine1 1: erro de rastreamento baseline
vs cerimônia
ax = axes[0]
for k in range(K):
    ax.plot(hist_baseline['abs_err'][:, k],
color=colors[k], alpha=0.4, linewidth=1)
    ax.plot(hist_ceremony['abs_err'][:, k],
color=colors[k], linewidth=1.6,
linestyle='--')
ax.axvspan(ceremony_window[0],
ceremony_window[1], color='purple',
alpha=0.15, label='janela de cerimônia')
ax.set_title('Erro de rastreamento |crença
- real|: sólido fino = baseline, tracejado
= com cerimônia')
ax.set_ylabel('|erro|')
ax.legend()
ax.grid(True, alpha=0.3)

# Paine1 2: HRV sintético e período
refratário derivado
ax = axes[1]
ax.plot(hrv_series, color='crimson',
label='HRV (RMSSD, sintético-placeholder)')
ax.set_ylabel('HRV (ms)', color='crimson')
ax2 = ax.twinx()
ax2.plot(refractory_from_hrv, color='navy',
alpha=0.6, label='período refratário
derivado')

```

```

ax2.set_ylabel('Período refratário
(passos)', color='navy')
ax.set_title('HRV sintético -> período
refratário (substituir por dado real)')
ax.grid(True, alpha=0.3)

# Painel 3: ignições, refratário fixo
ax = axes[2]
ign_steps =
np.where(hist_fixed_ref['ignition'])[0]
for t in ign_steps:
    k = int(hist_fixed_ref['winner'][t])
    ax.scatter(t, k, color=colors[k], s=20)
ax.set_yticks(range(K));
ax.set_yticklabels(labels)
ax.set_title(f'Ignições – refratário FIXO
({refractory_period_default} passos) –
total: {len(ign_steps)}')
ax.grid(True, alpha=0.3)

# Painel 4: ignições, refratário via HRV
ax = axes[3]
ign_steps2 =
np.where(hist_hrv_ref['ignition'])[0]
for t in ign_steps2:
    k = int(hist_hrv_ref['winner'][t])
    ax.scatter(t, k, color=colors[k], s=20)
ax.set_yticks(range(K));
ax.set_yticklabels(labels)
ax.set_xlabel('Tempo (passos)')
ax.set_title(f'Ignições – refratário via
HRV (8 a 30 passos, variável) – total:
{len(ign_steps2)}')
ax.grid(True, alpha=0.3)

```

```
plt.tight_layout()
plt.savefig('/home/claude/active_inference_
workspace/ceremony_hrv_experiment.png',
dpi=150)
print("\nArquivo gerado:
ceremony_hrv_experiment.png")
```